



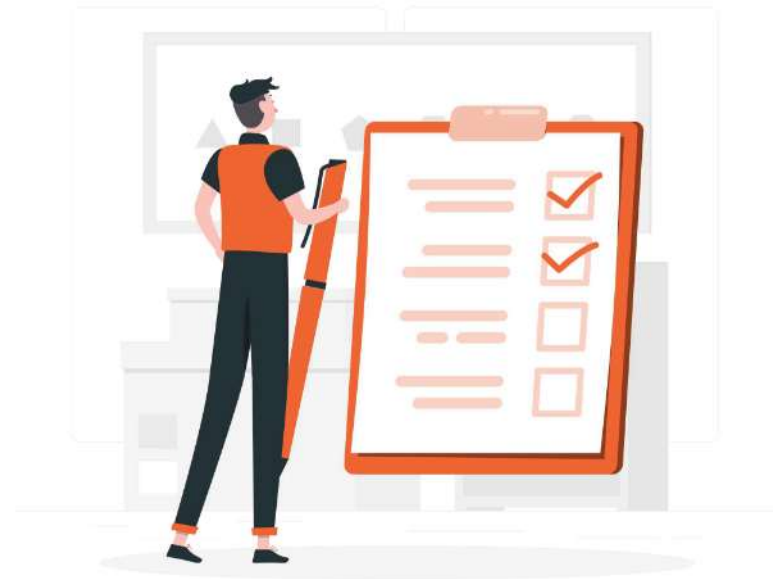
LLM Foundation Model





Today's Agenda

1. NLP Foundations
2. Inside the LLM: Transformers
3. Azure OpenAI Models
4. Context Window Concepts
5. How to Choose the Right Model





Learning Outcome

1. Identify fundamental concepts in NLP
2. Understand how Transformer models work
3. Utilize Azure OpenAI models & their core capabilities
4. Manage context window limitations
5. Select the right model by balancing quality, speed, and cost





NLP Foundations



How machine understand us?

Natural Language Processing

Natural Language Processing (NLP) is a branch of Artificial Intelligence that gives machines the ability to **read**, **process** and **derive** meaning from human languages.

The core challenge: human language is messy, ambiguous, context dependent, ect.



I love my cat



{ "I" = 1, "love" = 2, "my" = 3, "cat" = 4 }



Encoded Word = [1, 2, 3, 4]

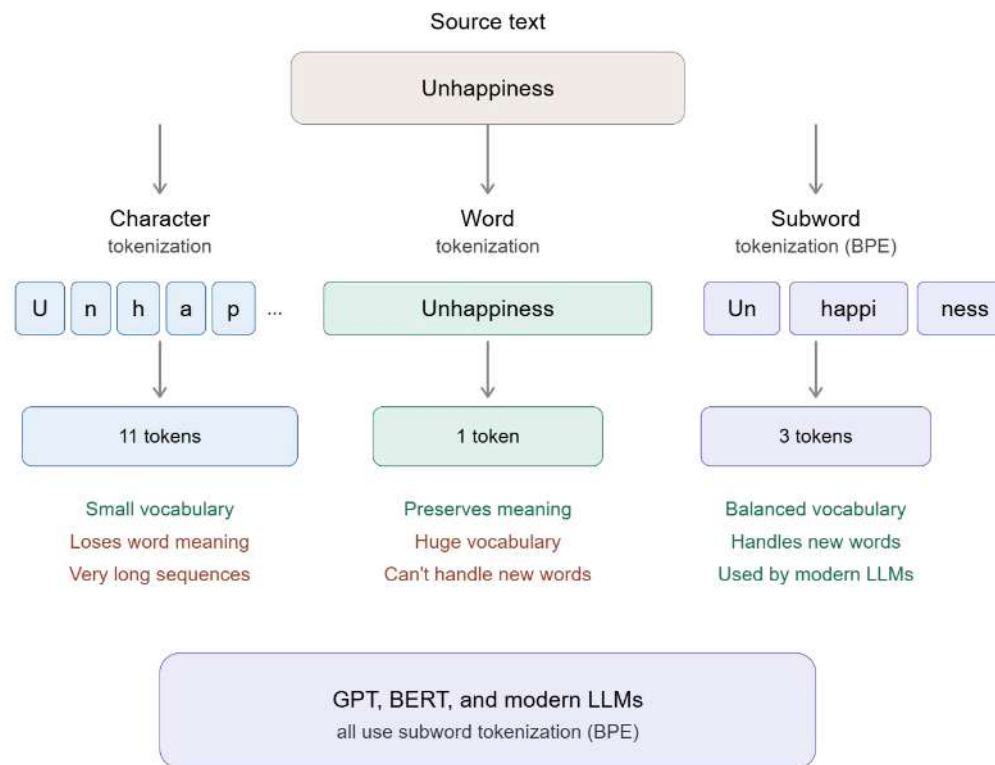
How machine understand us?

Tokenization

The very first step in any NLP system is **breaking down our sentence** into smaller units and **transforming them into numbers**. This process is called Tokenization.



Tokenization



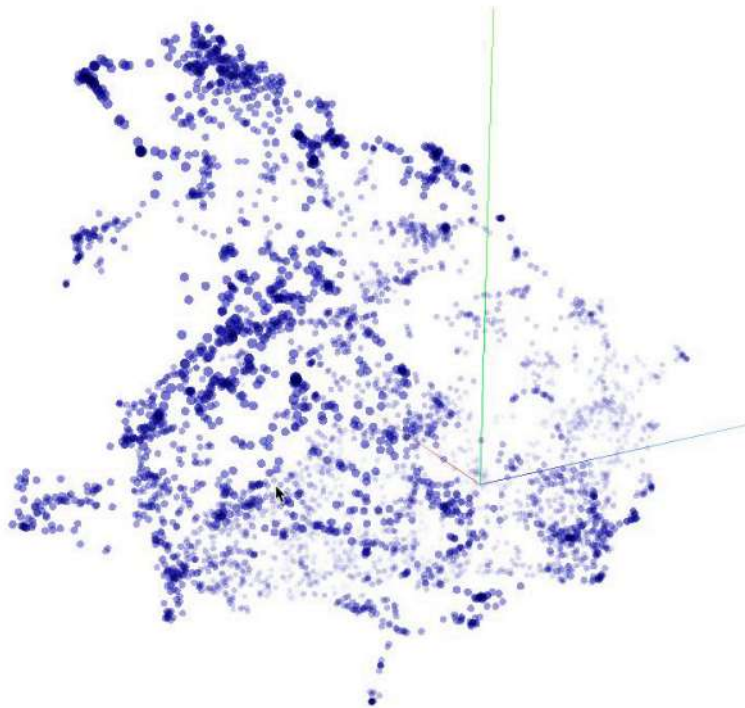


How machine understand us?

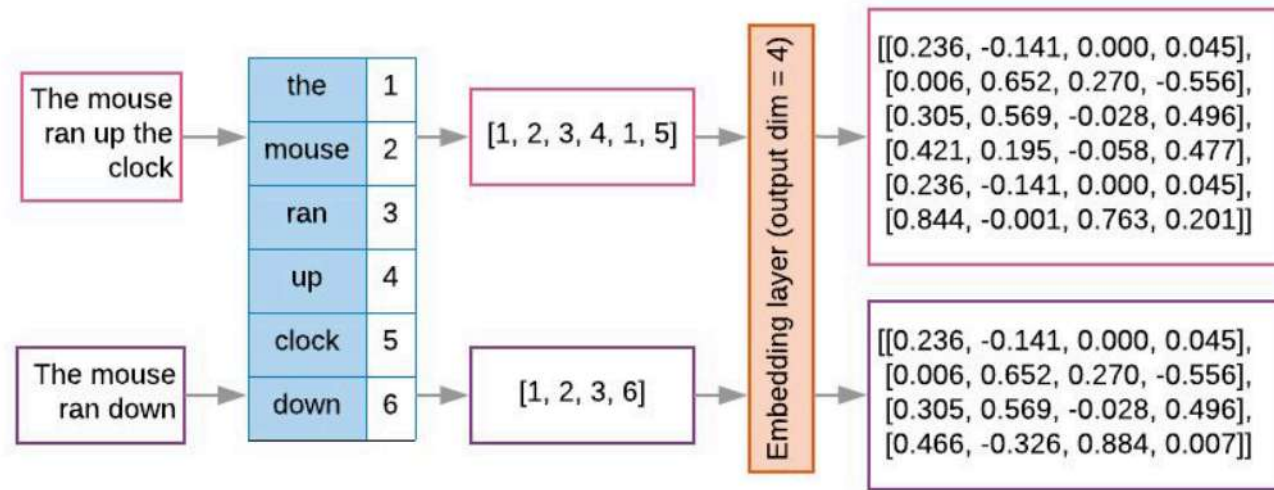
Embeddings

Embeddings are a type of word representation that allows words with similar meanings to have **similar vector representations**.

They map text into a vector space (semantic space) where **similar items are placed closer together**, allowing models to understand relationships.



Embeddings

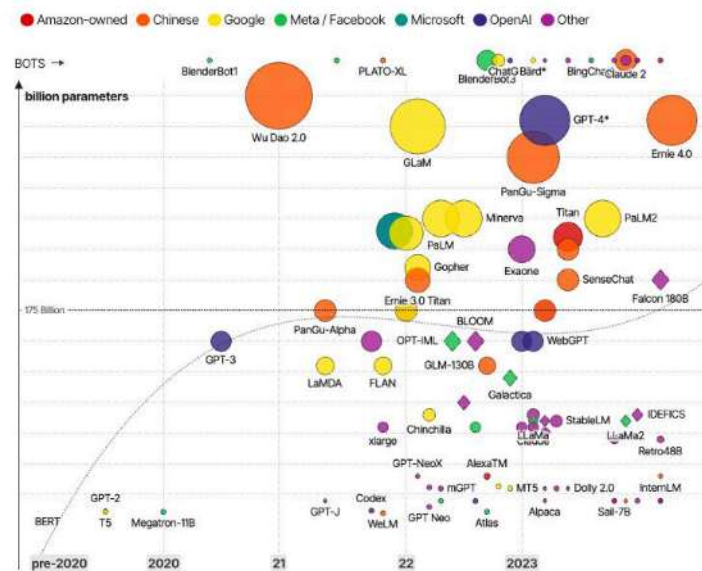




Inside the LLM: Transformers



The Rise and Rise of A.I. Large Language Models (LLMs) & their associated bots like ChatGPT



How LLM works?

Large Language Models

A Large Language Model (LLM) is a **type of NLP model** that is trained on vast datasets to **understand, generate, and manipulate** human language.

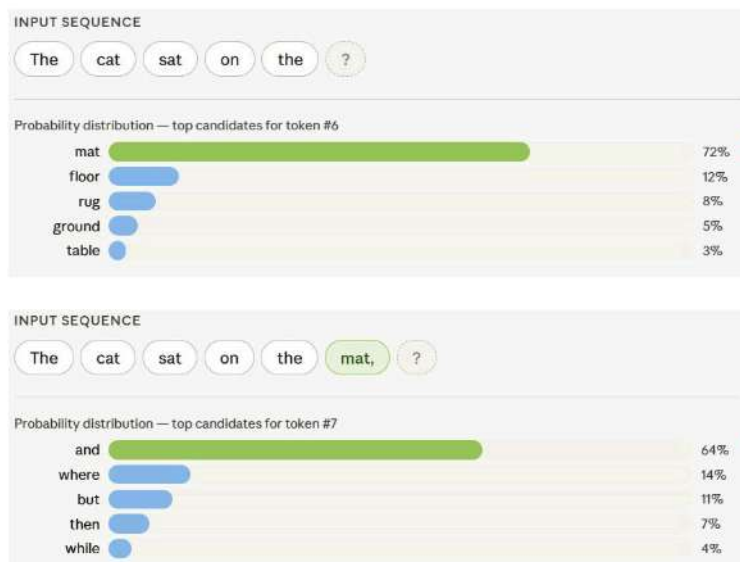
Large refers to both the training dataset and the number of parameters.

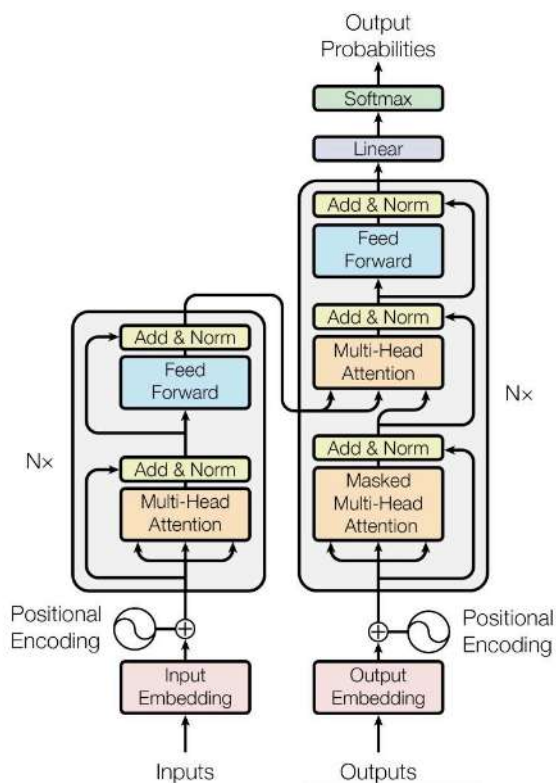


How LLM works?

How LLM Generate Text?

Text generation in LLMs is **autoregressive**. This means the model **generates one token at a time**, and each **new token is appended to the input** to help predict the subsequent one.





How LLM works?

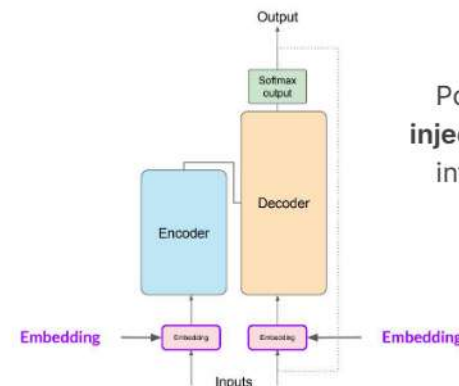
Transformer Architecture

The original Transformer paper (Attention is All You Need) introduced a dual-structure:

- **Encoder:** processes the input sequence to **create a rich numerical representation** (context) of the data.
- **Decoder:** uses the encoder's representation and previous outputs to **generate a target sequence**.

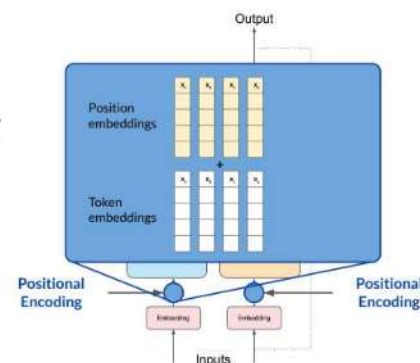


Deep Dive: Inside the Transformer

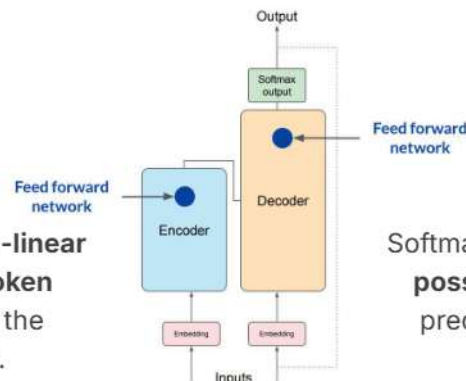
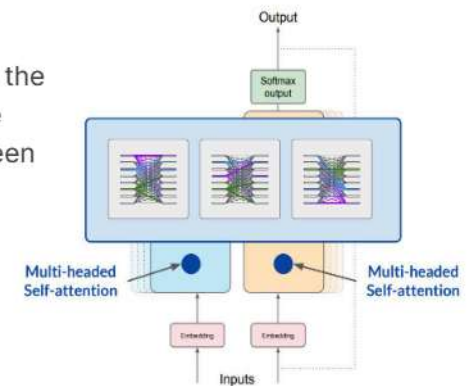


Converts words into **high-dimensional vectors** where **similar words are closer together** in vector space.

Positional encoding **injects sequence order** information of each token.

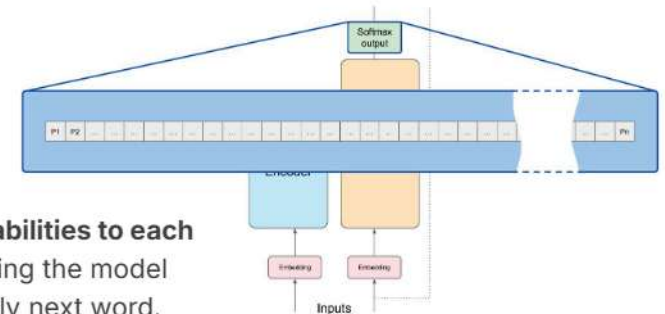


Self-attention allows the model to **capture relationships** between words.

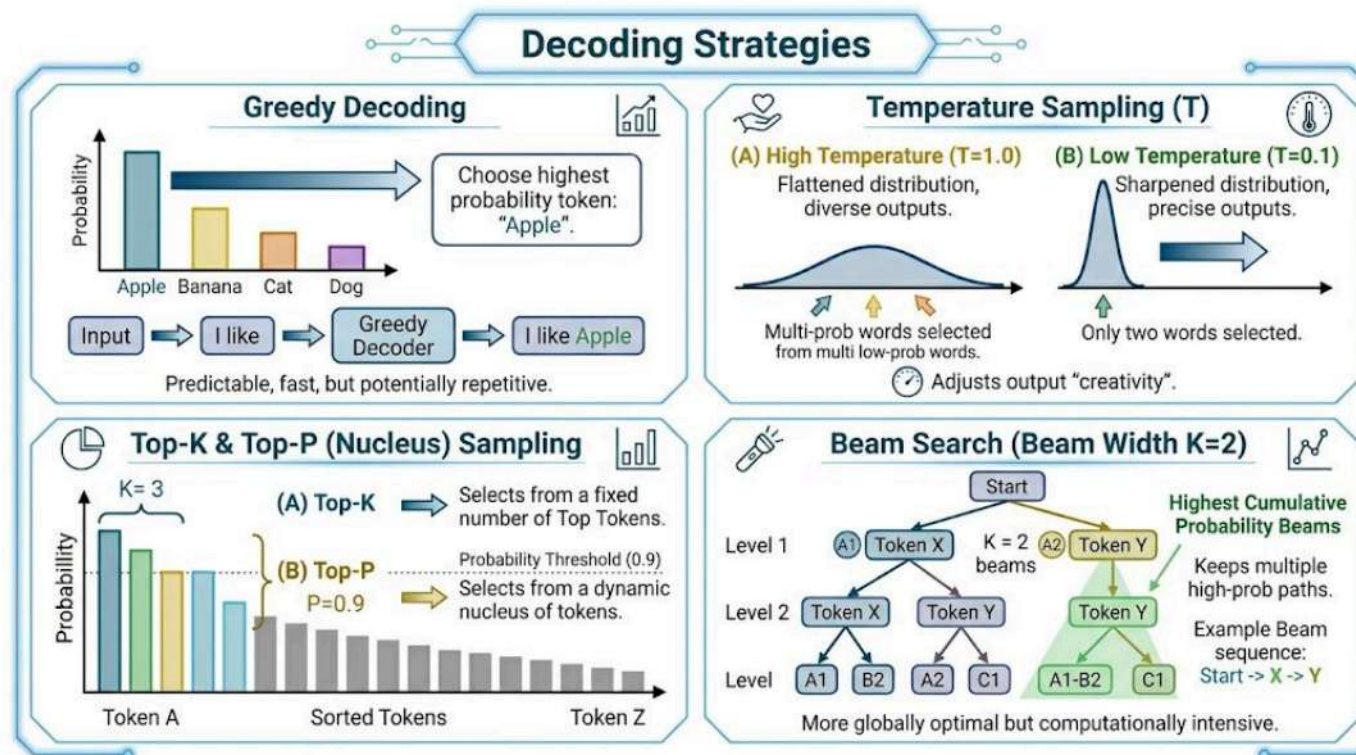


Applies a **position-wise non-linear transformation** to each token independently, enriching the representation further.

Softmax **assigns probabilities to each possible token**, helping the model predict the most likely next word.



How LLMs Pick the Next Token?





Azure OpenAI Models



What is Azure OpenAI?

Azure OpenAI Service provides **REST API access to OpenAI's** powerful models with added **enterprise security, compliance,** and **regional deployment** via Microsoft Azure.

Enterprise Security

Private endpoints, VNet integration, role-based access (RBAC), Azure AD authentication

Global Deployment

Deploy models in 30+ Azure regions; choose data residency for compliance requirements

Responsible AI

Built-in content filtering, abuse monitoring, and alignment with Azure AI safety standards

Managed Scaling

Provisioned Throughput Units (PTU) for guaranteed capacity or pay-per-token standard tier



Sample Code in Python

```
# pip install openai

from openai import AzureOpenAI

client = AzureOpenAI(
    azure_endpoint = "https://<your-resource>.openai.azure.com/",
    api_key        = "YOUR_AZURE_OPENAI_KEY",
    api_version    = "2024-02-01",
)

response = client.chat.completions.create(
    model = "gpt-4o",          # your deployment name
    messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user",   "content": "Explain LLMs in 3 bullets."},
    ],
    max_tokens = 512,
)

print(response.choices[0].message.content)
```

AzureOpenAI()

Use AzureOpenAI client, not OpenAI — requires endpoint, key, and api_version

model parameter

Pass your Azure deployment name (not the model name — check Azure portal)

api_version

Use '2024-02-01' or latest stable GA version. Preview versions add features but may change

Streaming

Add stream=True to receive tokens incrementally; iterate over response chunks



Parameters in LLMs

Beyond temperature: every parameter that controls Azure OpenAI generation behavior.

Parameter	Range	Description
<code>temperature</code>	0.0 – 2.0	Controls randomness. 0 = deterministic, 1+ = creative.
<code>top_p</code>	0.0 – 1.0	Nucleus sampling. Alternative to temperature.
<code>max_tokens</code>	1 – 128K	Hard limit on output length. Prevents runaway.
<code>frequency_penalty</code>	0.0 – 2.0	Reduces word repetition. 0.5–1.0 for essays.
<code>presence_penalty</code>	0.0 – 2.0	Encourages new topics. Good for brainstorming.
<code>stop</code>	up to 4	Stop generation at specific strings/tokens.





Parameters in LLMs

Use Case	temperature	top_p	freq_penalty	pres_penalty
 Factual Q&A	0	1.0	0	0
 Creative Writing	0.8	0.9	0.5	0.3
 Code Generation	0.2	0.95	0	0
 Brainstorming	1.2	1.0	0	1.5

⚡ Pro tip: Start with a preset, then adjust ONE parameter at a time based on output quality.



Other LLM Providers

Provider	Top Models	Key Strength	Watch Out For	Access
OpenAI / Azure	GPT-4o, o3, o4-mini	Best overall quality, huge ecosystem, enterprise SLA via Azure	Most expensive; proprietary, no self-hosting	API / Azure
Google DeepMind	Gemini 2.5 Pro/Flash	Longest context (1M tokens), native multimodal, deep GCP integration	API latency can vary; pricing complex for high volume	AI Studio / Vertex AI
Anthropic	Claude 4 Opus/Sonnet	Best instruction following, safest outputs, 200K context window	No image generation; smaller ecosystem vs OpenAI	API / AWS Bedrock
Meta (Open Source)	Llama 3.3, 4 Scout/Maverick	Free to use, self-host on your infra, no data leaves your environment	Requires GPU infra; smaller out-of-the-box quality vs. frontier	Download / HuggingFace
Mistral AI	Mistral Large, Codestral	Best open-weight code model; European data sovereignty compliance	Smaller community; fewer multimodal capabilities	API / Azure / Self-host
Cohere	Command R+, Embed v3	Best-in-class RAG & retrieval; enterprise focus, strong embeddings	Less general-purpose than GPT-4 tier models	API / AWS / Azure



Context Window Concepts



What is a Context Window?

The context window is the **total number of tokens** an LLM can process at one time. It's the **model's working memory**.



Token Size Reference

1 word $\approx$ 1–2 tokens "Hello" = 1 token	1 page (500 words) $\approx$ ~750 tokens A standard A4 document page	1 novel $\approx$ ~130K tokens Harry Potter book 1	1M tokens $\approx$ ~750K words ~7 full-length novels
--	---	---	--



Managing the Context Window

01 Prompt Compression

Remove filler words, repeated context, and verbose instructions. Use structured formats (JSON, bullet lists) instead of prose descriptions. Measure tokens with tiktoken before sending.

💡 Rule of thumb: compress system prompt to < 500 tokens

02 Chunking & Summarization

Split long documents into chunks (500–1000 tokens). Process each chunk separately, then pass summaries to final synthesis step. Prevents "lost in the middle" degradation.

💡 Use recursive summarization for very long corpora

03 Retrieval-Augmented Generation (RAG)

Instead of fitting entire knowledge bases in context, retrieve only the top-K relevant chunks using semantic search (embeddings). Inject only what's needed into each request.

💡 Keeps context lean; enables infinite-scale knowledge

04 Sliding Window & Memory

For long conversations, keep only recent N turns in context. Summarize older history into a compressed memory block. Use structured memory stores for persistent entity tracking.

💡 LangChain ConversationSummaryBufferMemory implements this



Available Models on Azure OpenAI

Model	Context Window	Best For	Tier
 GPT-4o	128K tokens	Multimodal tasks — text, images, audio, vision	Standard / PTU
 GPT-4o mini	128K tokens	Cost-efficient tasks, high-volume pipelines	Standard
 o3	200K tokens	Complex reasoning, multi-step math & coding	Standard
 o4-mini	200K tokens	Fast reasoning at lower cost than o3	Standard
 GPT-4 Turbo	128K tokens	Legacy enterprise workloads, long documents	Standard / PTU
 text-embed-3	8K tokens	Semantic search, RAG pipelines, clustering	Standard





How to Choose the Right Model



The 4 Dimensions of Model Selection

★ Quality	⚡ Speed	\$ Cost	🔒 Constraints
How accurate, coherent, and capable is the model on your specific task? Measured by benchmark performance and real-task evals.	How fast does the model respond? Latency matters for real-time applications. Measured in tokens/second or time-to-first-token (TTFT).	What does it cost per 1M input and output tokens? Factor in expected volume to estimate monthly spend. Output tokens cost 2–4× more.	Non-functional requirements: data residency, compliance, modality (vision, audio), open vs closed source, self-hosting needs.
Key Metrics	Key Metrics	Key Metrics	Key Metrics
MMLU HumanEval Internal evals Human preference	TTFT TPS (tokens/sec) P50/P99 latency Streaming support	\$/1M input tokens \$/1M output tokens Context cache pricing Batch discounts	Data sovereignty GDPR / HIPAA Multimodal support Open-source



Model Selection Decision Framework

START: Define Use Case

Q1: Does quality matter most?

☒ YES → GPT-4o or Claude Opus

NO → consider speed & cost

Q2: Need fast / real-time?

☒ YES → GPT-4o mini or Gemini Flash

NO → batch tier OK

Q3: Cost sensitive?

☒ YES → Llama 3 or Mistral

NO → use premium tier

Scenario-Based Recommendations

Customer Chatbot (high volume)	GPT-4o mini / Gemini Flash	Fast, cheap, good enough quality	Speed + Cost
Legal / Medical Document Analysis	GPT-4o / Claude Sonnet	Maximum accuracy; long context support	Quality
Code Generation & Review	GPT-4o / Codestral	Top HumanEval scores; code-specialized	Quality + Code
RAG Knowledge Base Q&A	GPT-4o mini + Cohere Embed	Cheap inference + best-in-class retrieval	Cost + Retrieval
On-Prem / Data Sovereign	Llama 3.3 / Mistral Large	Self-hosted; no data leaves your infra	Compliance
Multimodal (image+text)	GPT-4o / Gemini 2.5 Pro	Native vision + text in single model call	Multimodal



Model Pricing Comparison

(per 1M tokens, 2026)

Model	Provider	Input	Output	Context	Best For
 GPT-5.3	Azure/OpenAI	\$1.75	\$14.00	128K	Premium tasks, multimodal
 GPT-4.1	Azure/OpenAI	\$2.00	\$3.5	1M	High-vol chatbots, APIs
 o3	Azure/OpenAI	\$2.00	\$8.00	200K	Complex reasoning, math
 Claude Sonnet 4	Anthropic	\$3.00	\$15.00	200K	Documents, long context
 Gemini 3.1 Flash	Google	\$0.25	\$1.5	1M	Speed, multimodal, scale
 Gemini 3.1 Pro	Google	\$2.00	\$12.00	1M	Complex, 1M ctx tasks
 Llama 3.3 70B	Self-hosted	~\$0	~\$0	128K	Private/on-prem workloads
 Mistral Large	Mistral/Azure	\$0.50	\$1.50	256K	EU compliance, code



Guided Hands-on

Open in Google Colab ↗





Assignment





Assignment: AI Project Planning

Use the worksheet template on the next slides & present your plan to the class at the end.

01 	02 	03 	04 	05 
Project Scope & Goals	Model Selection	Prompt Design	Token Estimation	Cost Calculation
Define what your AI system does, who uses it, and what success looks like	Choose the right LLM using the Quality → Speed → Cost → Constraints framework	Write your system prompt, define context, user message format, and expected output	Use the OpenAI Tokenizer to measure your prompts and calculate real token counts	Estimate monthly API spend based on volume x tokens x price per 1M

<https://platform.openai.com/tokenizer>



STEP 01: Define Your Project Scope & Goals

Answer these 5 questions together. Be specific, your answers will drive every decision in the next steps.

1

What does your AI system do?

e.g. A customer support chatbot for an e-commerce platform that answers order questions in Bahasa Indonesia

Describe the core function in 1–2 sentences

2

Who are the users?

e.g. End customers (non-technical), approx. 5,000 active users/day

Role, technical level, expected volume

3

What does 'success' look like?

e.g. Answer accuracy > 90%, response in < 2 seconds, reduces support tickets by 40%

Define 2–3 measurable success criteria

4

What are your hard constraints?

e.g. Data must stay in Indonesia (data residency), budget < \$500/month, must support Bahasa Indonesia

Compliance, language, budget ceiling, latency SLA

5

What data will you feed the model?

e.g. Product catalog (5,000 items), FAQ documents (120 pages), order history (per-session)

List data sources and rough sizes





STEP 02: Choose the Right Model

Use your answers from Step 01 to fill in this decision matrix. Score each dimension 1–3 for your use case, then pick a model.

Dimension	Your Requirement	Score (1–3)	Drives Toward
Quality	How accurate must answers be? (legal/medical = high; FAQs = medium)	3 = critical / 1 = OK if fast	3 → GPT-4o / Claude 1 → mini / Flash
Speed	Max acceptable response time? (real-time chat vs. batch report)	3 = < 1 sec / 1 = minutes OK	3 → Flash / mini 1 → o3 / Opus OK
Cost	Monthly budget ceiling? (startup = tight; enterprise = flexible)	3 = very tight / 1 = flexible	3 → Llama / mini 1 → premium OK
Constraints	Data residency / compliance? (GDPR, HIPAA, on-prem, language support)	3 = strict / 1 = none	3 → Llama / Azure 1 → any provider
Our Group's Model Choice: Write model name here (e.g. GPT-4o mini) Reason: 1 sentence justification			



Batas Pengumpulan: Hari ini, pukul 23.59 WIB
Pengumpulan melalui: Google Classroom



